

Documentação da Rota de Dashboard Administrativo

Visão geral

A rota:

GET /api/dashboard

É responsável por fornecer ao painel administrativo os dados brutos necessários para montar os indicadores, gráficos e listas do dashboard.

Ela não realiza filtros analíticos, cálculos de métricas ou agregações específicas. Essa responsabilidade fica no front-end, mantendo a API simples, reutilizável e fácil de evoluir.

A rota segue o mesmo padrão de autenticação e validação de usuário já utilizado no projeto: sessão via Supabase, busca do perfil na tabela `usuarios`, validação de status e autorização por perfil.

Objetivo da rota

Retornar, para usuários administradores autenticados, os dados completos de:

clientes

ordens_servico

vendedores

sempre restritos à mesma `conta_id` do usuário logado.

Esses dados são utilizados no front-end para construir:

- total vendido;
- quantidade de OS abertas;
- OS atrasadas;
- OS prontas para retirada;
- clientes novos;
- ticket médio;
- gráficos de vendas;
- gráfico de OS por status;
- ranking de vendedores;

- últimas vendas;
- listas operacionais rápidas.

Esses indicadores estão alinhados com o escopo do dashboard administrativo definido para o sistema.

Método e endpoint

GET /api/dashboard

Autenticação

A rota exige usuário autenticado por sessão válida do Supabase.

Internamente, a validação é feita com:

```
supabase.auth.getUser()
```

Caso não exista usuário autenticado, a rota retorna:

```
{  
  "error": "Usuário não autenticado."  
}
```

Status HTTP:

401 Unauthorized

Regras de autorização

Depois da autenticação, a rota consulta a tabela:

usuarios

e busca o perfil correspondente ao ID do usuário autenticado.

Validações realizadas

1. Perfil existente

Se o usuário autenticado não possuir registro na tabela `usuarios`, a resposta será:

```
{  
  "error": "Perfil de usuário não encontrado."  
}
```

Status HTTP:

404 Not Found

2. Usuário ativo

A rota verifica:

```
usuario.status === "ativo"
```

Se o usuário estiver inativo ou bloqueado:

```
{  
  "error": "Seu acesso está inativo ou bloqueado."  
}
```

Status HTTP:

403 Forbidden

3. Perfil administrador

A rota permite acesso apenas quando:

```
usuario.role === "admin"
```

Se o usuário possuir outro perfil:

```
{  
  "error": "Acesso permitido apenas para administradores."  
}
```

Status HTTP:

Escopo dos dados retornados

Todos os dados consultados respeitam a conta do usuário autenticado:

```
.eq("conta_id", contald)
```

Isso impede que um administrador visualize dados de outra conta.

As tabelas carregadas são:

clientes

ordens_servico

vendedores

Todas utilizam:

```
.select("*")
```

A escolha por `select ("*")` torna a rota mais simples e evita retrabalho constante quando novos campos forem adicionados às tabelas.

Consultas realizadas

A rota executa as três consultas em paralelo com `Promise.all()`:

```
const [clientesResult, ordensServicoResult, vendedoresResult] =
```

```
await Promise.all([
```

```
  supabase.from("clientes").select("*").eq("conta_id", contald),
```

```
  supabase
```

```
    .from("ordens_servico")
```

```
    .select("*")
```

```
    .eq("conta_id", contald),
```

```
supabase.from("vendedores").select("*").eq("conta_id", contaid),
]);
```

Essa abordagem reduz o tempo total da resposta, pois as buscas são feitas simultaneamente.

Resposta de sucesso

Status HTTP:

200 OK

Body:

```
{
  "success": true,
  "data": {
    "clientes": [],
    "ordens_servico": [],
    "vendedores": []
  }
}
```

Estrutura da resposta

Campo **success**

Indica que a requisição foi concluída corretamente.

```
"success": true
```

Campo **data**

Agrupar os conjuntos brutos de dados necessários para o dashboard:

```
"data": {  
  "clientes": [],  
  "ordens_servico": [],  
  "vendedores": []  
}
```

Tratamento de erros internos

Se alguma das consultas ao banco falhar, o erro é lançado e capturado pelo **catch**.

A resposta será:

```
{  
  "error": "Erro interno ao carregar os dados do dashboard."  
}
```

Status HTTP:

500 Internal Server Error

O erro técnico também é registrado no servidor:

```
console.error("Erro ao carregar dashboard:", error);
```

Responsabilidades do front-end

A rota retorna dados brutos. O front-end deve montar:

- filtros de período;
- filtros por vendedor;
- somatórios;
- ticket médio;
- cálculo de OS atrasadas;
- vendas por dia ou mês;

- distribuição por status;
- ranking de vendedores;
- listas resumidas.

Essa decisão reduz acoplamento e acelera desenvolvimento, pois a rota não precisa ser alterada sempre que o dashboard ganhar novo gráfico ou novo filtro.

Fluxo completo da rota

1. Recebe GET /api/dashboard
2. Cria cliente Supabase server-side
3. Obtém o usuário autenticado
4. Verifica se existe sessão
5. Busca o perfil na tabela usuarios
6. Confirma se o usuário está ativo
7. Confirma se o perfil é admin
8. Captura a conta_id do usuário
9. Busca clientes, ordens_servico e vendedores da mesma conta
10. Retorna os dados brutos em JSON

Exemplo de uso no front-end

```
const response = await fetch("/api/dashboard", {  
  
  method: "GET",  
  
  cache: "no-store",  
  
});
```

```
const payload = await response.json();
```

Resumo técnico

Item	Definição
Endpoint	/api/dashboard
Método	GET
Autenticação	Obrigatória
Perfil permitido	admin
Fonte da sessão	Supabase Auth
Tabelas retornadas	clientes, ordens_servico, vendedores
Filtro aplicado	conta_id
Seleção	select("*")
Filtros analíticos	Front-end
Agregações	Front-end
Resposta	JSON

